

一、多模态大模型与具身智能 00:30

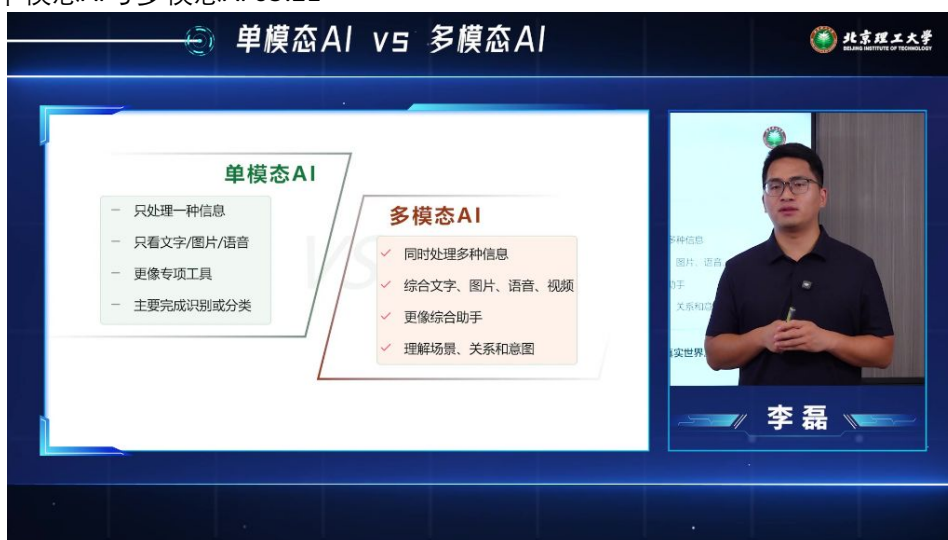
1. 什么是多模态 02:24

1) 多模态 02:27



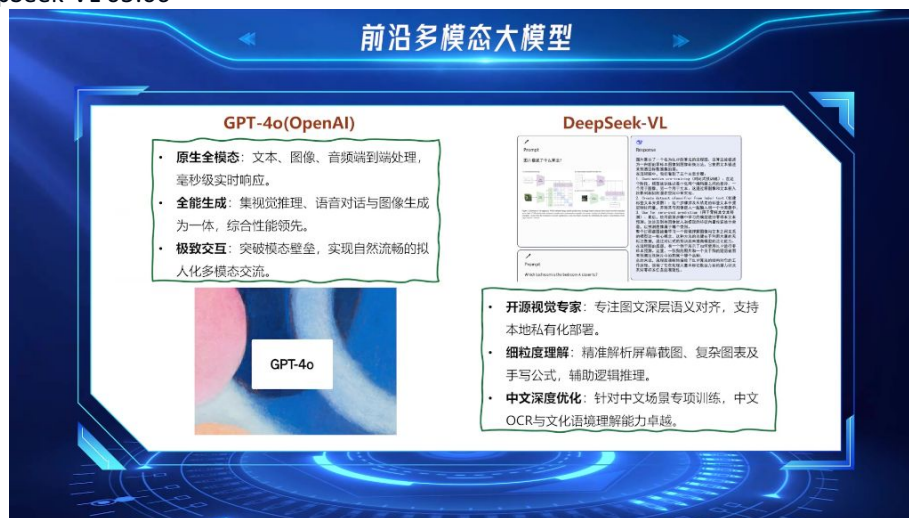
- **多模态的定义:** 让同一个AI系统综合理解文本、图像、语音、视频、传感器等多种信息形式。
 - 人类理解世界并非只靠文字，而是视觉、听觉、触觉等多种信息同时涌入大脑。AI也需要类似的能力。
- **多模态的核心:** 不是简单的“多输入”，而是**对齐和推理**。
 - **对齐:** 模型需要将不同模态的信息在语义层面关联起来。例如，“红色的苹果”这个文本，要能关联到图像中红色圆形的物体，以及咬下去脆甜的口感。
 - **推理:** 在对齐的基础上，融合多种信息形成统一的判断。
- **常见误区:** 很多人认为多模态就是给模型增加几个输入接口，这是错误的。真正的多模态在于模型内部对信息的深度融合与理解。

2) 单模态AI与多模态AI 03:21

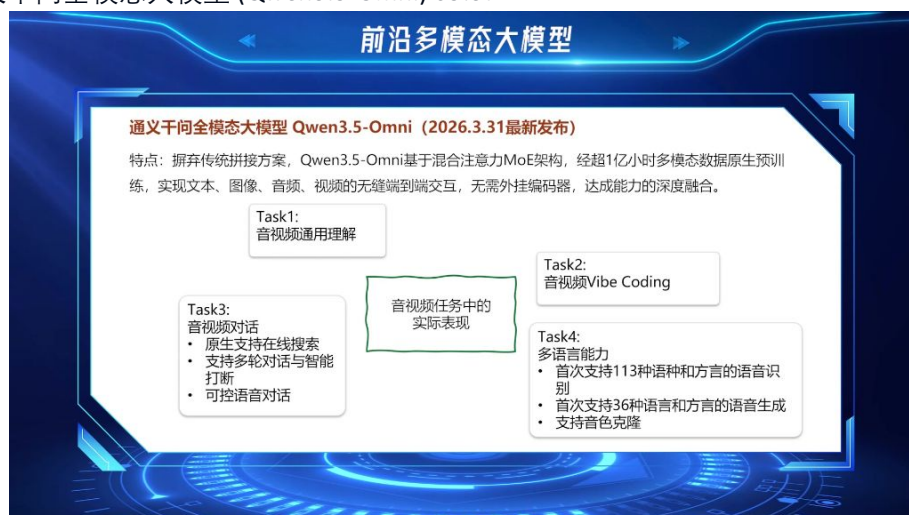


- **单模态AI:** 只处理一种信息（如文字、图片或语音），本质上是专项工具，完成识别或分类任务就结束。
 - 例子：早期的人脸识别、语音转文字。

- **多模态AI:** 同时处理多种信息，是综合助手，能理解场景、关系和意图。
 - 例子：拍一张冰箱内部的照片，问“我晚上能做什么菜？”。模型不仅要识别出鸡蛋、西红柿，还要结合“晚上做菜”的上下文语境，推荐“番茄炒蛋”。
- **多模态的价值:** 一句话概括——**理解更接近真实世界的复杂性。**
- 3) 前沿多模态大模型 04:30
 - **GPT-4o (OpenAI)**
 - **核心突破:** 原生全模态。
 - 从输入端就将文本、图像、音频融合在一起，进行端到端处理，实现毫秒级实时响应。
 - 对比：以往的多模态模型是“拼接”的（视觉用一个模型，音频用一个模型，最后塞进大语言模型），信息损失大。GPT-4o将所有模态揉在一起，交互更自然。
 - **能力特点:** 集视觉推理、语音对话与图像生成为一体，综合性能领先。
 - **DeepSeek-VL 05:06**



-
- **核心定位:** 开源视觉专家，专注图文深层语义对齐，支持本地私有化部署。
- **能力特点:**
 - **细粒度理解:** 能精准解析屏幕截图、复杂图表、手写公式，辅助逻辑推理。
 - **中文深度优化:** 针对中文场景专项训练，中文OCR和文化语境理解能力优于很多国外模型。
- **通义千问全模态大模型 (Qwen3.5-Omni) 05:37**



-
- **核心突破:** 彻底抛弃外挂编码器的拼接方案。

- 采用**混合注意力MoE架构**，经过超1亿小时多模态数据原生预训练，实现文本、图像、音频、视频的无缝端到端交互。
- **关键能力一：音视频通用理解**
 - 输入音视频，能生成结构化、带时间戳的准确描述（caption）。
 - 能识别说话者的口音（如北美口音）和**情绪状态**（如开心、不耐烦）。
- **关键能力二：音视频Vibe Coding**
 - 根据音视频指令直接生成网页代码，无需复杂的文字描述和参考图。
 - 例子：对着视频说“希望小蛇进入红框区域时变为春日主题、黄色色调”，模型能直接理解并生成对应代码，精度高，交互门槛低。

2. 具身智能 07:00

1) 什么是具身智能 07:22



- **核心定义**: 具身智能是指拥有物理或虚拟“身体”的AI形态。
 - 载体不限于机器人，还包括自动驾驶汽车、无人机、游戏中的NPC等。
- **与普通AI的本质区别**:
 - **普通AI**: 主要“回答问题”，例如告诉你怎么吃苹果的步骤。
 - **具身智能**: 要“完成动作”，例如把苹果从桌面上拿给你。



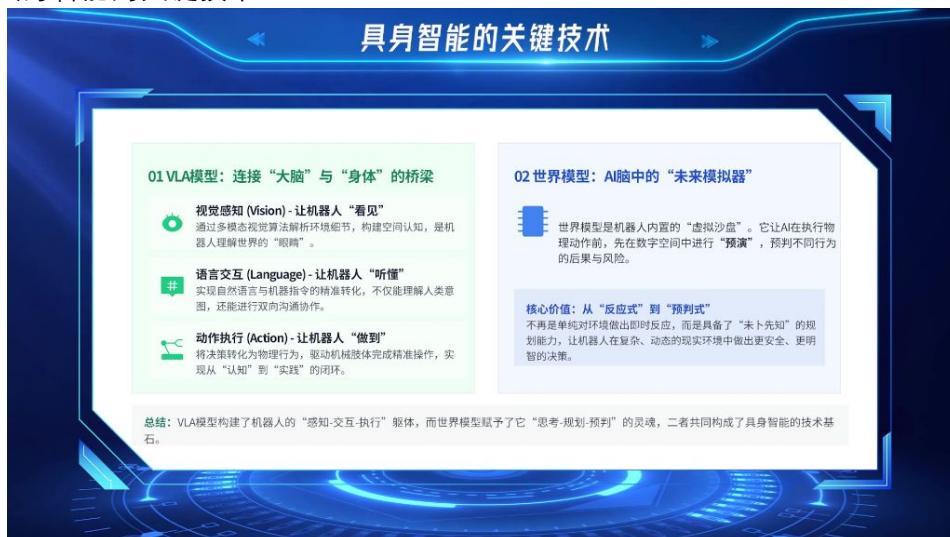
- **核心流程（五步闭环）**:
 - **感知**: 通过传感器（视觉、听觉、触觉等）获取环境信息。
 - **理解**: 对感知信息进行理解，构建对环境的认知。
 - **规划**: 根据目标与约束，制定行动计划（路径、策略等）。
 - **行动**: 执行动作，影响环境，完成任务。

- 反馈: 接收环境反馈, 评估结果, 持续学习与调整。
- 案例: 机器人拿苹果



- 任务: 把桌上的苹果拿给我。
- 拆解步骤:
 - 感知: 摄像头看到苹果和桌面, 识别出这是物理实体, 不是一幅画。
 - 理解: 判断苹果是否可抓取、有无遮挡、材质是否光滑等。
 - 规划: 计算苹果距离、机械臂抬升角度、抓取力度等。
 - 行动: 驱动关节、闭合夹爪, 拿起苹果。
 - 反馈: 若苹果滑了, 传感器立刻回传, 调整力度。
- 关键认知: 一个随手可做的动作, 背后是一个完整的闭环。具身智能的难点在于“如何做出来”, 而不是“算出来”。

2) 具身智能的关键技术 09:10



- VLA模型: 连接“大脑”与“身体”的桥梁
- **VLA模型** = Vision (视觉) + Language (语言) + Action (动作)。
 - **Vision (视觉感知)**: 让机器人“看见”。通过多模态视觉算法解析环境细节, 构建空间认知, 是理解世界的“眼睛”。
 - **Language (语言交互)**: 让机器人“听懂”。实现自然语言与机器指令的精准转化, 不仅能理解人类意图, 还能进行双向沟通 (如反馈“苹果太滑了, 我再试一次”)。
 - **Action (动作执行)**: 让机器人“做到”。将决策转化为物理行为, 驱动机械肢体完成精准操作, 实现从“认知”到“实践”的闭环。

- **总结:** VLA模型构建了机器人的“感知-交互-执行”躯体。
- **世界模型: AI脑中的“未来模拟器” 09:44**
 - **核心定义:** 世界模型是机器人内置的“虚拟沙盘”。
 - 让AI在执行物理动作前, 先在数字空间中进行“预演”, 预判不同行为的后果与风险。
 - **核心价值:** 从“反应式”到“预判式”的跨越。
 - **反应式(旧):** 碰到障碍才停, 撞了南墙才回头。
 - **预判式(新):** 具备“未卜先知”的规划能力, 在复杂、动态环境中做出更安全、更明智的决策。
 - **总结:** 世界模型赋予了机器人“思考-规划-预判”的灵魂。
- 3) **世界模型的应用场景 10:15**



- **自动驾驶:** 不仅识别行人和车辆, 还要预判他们下一步怎么动(直走还是突然跑), 提前减速或停车。
- **机器人抓取:** 不仅识别物体, 还要预测抓取时物体是否会滑落。
- **无人机飞行:** 在复杂环境中飞行, 预测飞行路径是否安全。
- **游戏/仿真:** 在虚拟世界中决策, 预测环境会如何变化。
- **核心原则:** 只会识别世界还不够, 真正行动之前必须预测后果。
- 4) **前沿案例: 人形机器人马拉松 10:54**



- **事件:** 2025年4月19日, 北京亦庄举办全球首次“人机共跑”半程马拉松。
- 天工队人形机器人以2小时40分42秒夺冠, 共6支队伍完赛。

- **技术剖析:**
 - **VLA模型:** 通过视觉感知跑道，通过动作控制迈步，实现自主导航。
 - **世界模型:** 预测每一步是否会滑倒，如何调整重心，确保在不平坦路面上稳定前进。
 - **进化速度:** 2025年还在“学习跑步”，2026年第二届已“学会跑步”——续航更稳、步态更顺、算法更成熟。
- ### 3. 多模态与具身智能的融合 11:50
- #### 1) GroundingDINO目标检测 12:59
- **融合的必要性:** 多模态大模型与具身智能必须融合，才能实现真正的AI智能体。
 - 仅有多模态大模型，只能在数字世界“纸上谈兵”，无法理解物理世界的真实规律（例如知道苹果会掉，但不知道加速度是多少）。
 - 仅有具身智能，缺乏强大的感知和理解能力，机器人就只是一个“四肢发达、头脑简单”的执行工具。
 - **融合后的逻辑:** 多模态大模型作为“超级感官”，将真实世界的非结构化信息（图像、语音等）转化为语义，告诉机器人“是什么、发生了什么”；具身智能作为“物理化身”，将决策转化为动作，并在互动中收集真实物理世界的实时反馈，反哺多模态模型，使其更懂真实世界。
 - **核心价值:** 这不是技术的简单叠加，而是从“数字大脑”到“完整生命”的跨越，是实现通用人工智能（AGI）的关键路径。
 - **环境准备 13:51**

教程：利用GroundingDINO实现目标检测任务

1. 环境准备

1.1 安装python

按Win+R，输入cmd打开命令行，输入 `python --version` 检查电脑上是否有python。

- 如果已经安装，会显示python版本号；
- 如果未安装，访问官网<https://www.python.org/downloads/release/python-3109/>，下载python，如下图，选择电脑合适的版本即可。



Version	Operating system	Description	File size	Signature	PGP	MDS checksum
Collected source tarball		Source release	24.9 MB	sigstore	SIG	2f4b108b3271151d838ac9981385c386
XZ compressed source tarball		Source release	18.7 MB	sigstore	SIG	463a4f2746238ac5e553342614c36418
macOS 64-bit universal2 installer	macOS	for macOS 10.9 and later	38.9 MB	CRT	SIG	432194b1ca530381c47731c1a0a8d31
Windows installer (64-bit)	Windows	Recommended	27.8 MB	CRT	SIG	81c157f1e1778910801a88c18a020ee
Windows installer (32-bit)	Windows		26.9 MB	CRT	SIG	1a1f1821077f8a01a0c1a0a8d319171
Windows help file	Windows		9.9 MB	CRT	SIG	c1a1f1821077f8a01a0c1a0a8d319171
Windows embeddable package (64-bit)	Windows		8.2 MB	CRT	SIG	c1a1f1821077f8a01a0c1a0a8d319171
Windows embeddable package (32-bit)	Windows		7.3 MB	CRT	SIG	8a1f1821077f8a01a0c1a0a8d319171

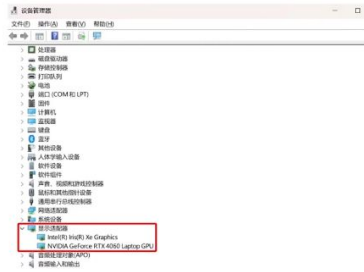
下载完成后按照提示安装即可，安装时需要勾选环境变量，如下图，其他部分按照默认配置即可。

现在由我来给大家讲一下如何利用grounding

- **安装Python:**
 - 按 Win + R，输入cmd 打开命令行，输入 `python --version` 检查是否已安装。
 - 若未安装，访问 Python官网 下载 Python 3.10.9。
 - 安装时务必勾选“Add Python to environment variables”（添加环境变量），其余按默认配置即可。

1.2 检查GPU

按Win+X, 选择“设备管理器”, 在其中找到“显示适配器”并点击查看。如果看到存在GPU, 则继续安装基于GPU的pytorch; 否则使用CPU训练。后续安装基于CPU的pytorch, 没有GPU仍然可以训练模型, 只是速度会更快。



1.3 安装CUDA Toolkit

在命令行输入nvidia-smi 确定CUDA版本号, 访问官网<https://developer.nvidia.com/cuda-toolkit-archive>和适合自己GPU的版本进行安装。可以按照默认配置安装。



检查GPU:

- 按 Win + X, 选择“设备管理器”, 找到“显示适配器”查看。
- 若存在GPU (如NVIDIA GeForce RTX 4060), 后续安装基于GPU的PyTorch; 否则只能使用CPU训练, 速度较慢但可行。

1.3 安装CUDA Toolkit

在命令行输入nvidia-smi 确定CUDA版本号, 访问官网<https://developer.nvidia.com/cuda-toolkit-archive>和适合自己GPU的版本进行安装。可以按照默认配置安装。



第三步, 如果电脑上存在GPU, 我们需要安装工具箱。

访问官网<https://git-scm.com/download/win>安装git, 可以按照默认配置安装。

1.5 安装 Visual Studio Build Tools

安装CUDA Toolkit:

- 在命令行输入 nvidia-smi 确定CUDA版本号。
- 访问 CUDA Toolkit Archive 找到适合自己GPU的版本, 按默认配置安装。

1.4 安装git

访问官网<https://git-scm.com/download/win>安装git, 可以按照默认配置安装。

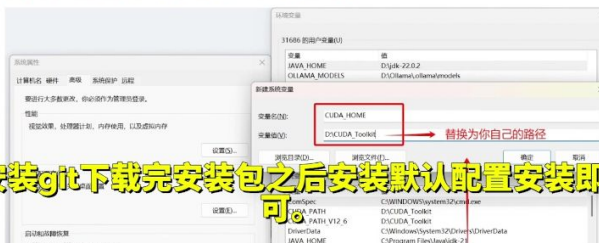
1.5 安装 Visual Studio Build Tools

访问官网<https://visualstudio.microsoft.com/zh-hans/downloads/> 下载Visual Studio 2022 Build Tools, 运行安装器, 在工作负载类型勾选“使用C++的桌面开发”, 右侧将节里勾选了MSVC v143和Windows 10/11 SDK; 安装完成即电脑。

按win+R, 输入cmd打开命令行, 依次检查以下内容:

- 输入python --version 检查python版本, 需要看到正确显示python版本;
- 输入pip --version 检查pip版本, 需要看到正确显示pip版本, pip在安装python时自动安装;
- 输入git --version 检查git版本, 需要看到正确显示git版本;
- 输入nvcc --version 检查CUDA Toolkit版本, 需要看到正确显示CUDA Toolkit版本;
- 输入echo %CUDA_HOME% 检查系统变量, 需要正确看到系统变量CUDA_HOME路径; 如果显示%CUDA_HOME%说明没有添加系统变量。

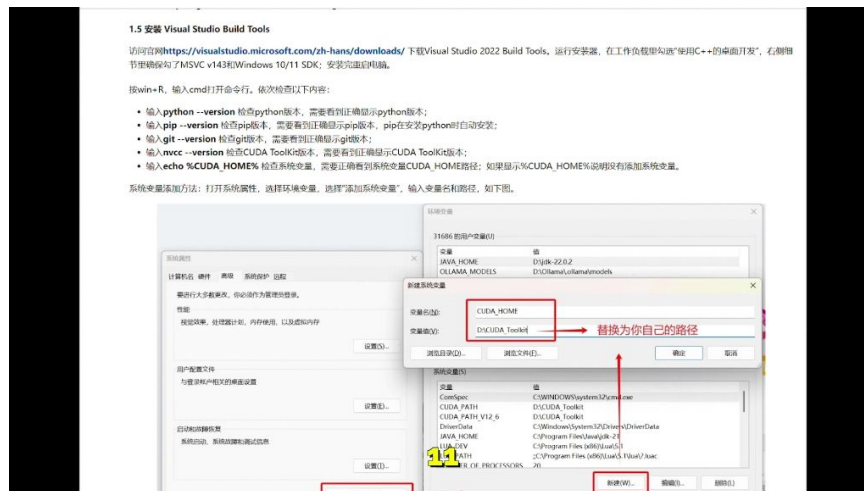
系统变量添加方法: 打开系统属性, 选择“高级系统变量”, 输入变量名称和路径, 如下图。



安装git下载完安装包之后安装默认配置安装即可。

安装Git:

- 访问 Git官网 下载安装包, 按默认配置安装。



○ 安装Visual Studio Build Tools:

- 访问 Visual Studio官网 下载 Visual Studio 2022 Build Tools。
- 运行安装器，在工作负载中勾选“使用C++的桌面开发”，右侧细节确保勾选“MSVC v143”和“Windows 10/11 SDK”。
- 安装完成后重启电脑。

○ 验证安装:

- 按 Win + R，输入 cmd 打开命令行，依次检查以下内容：
 - python --version: 正确显示Python版本。
 - pip --version: 正确显示pip版本（安装Python时自动安装）。
 - git --version: 正确显示Git版本。
 - nvcc --version: 正确显示CUDA Toolkit版本。
 - echo %CUDA_HOME%: 正确显示系统变量路径。

○ 添加系统变量:

- 若显示 %CUDA_HOME%，说明未添加系统变量。
- 打开“系统属性”→“环境变量”→在“系统变量”中点击“新建”。
- 变量名输入 CUDA_HOME，变量值输入CUDA Toolkit的安装路径（如 D:\CUDA_Toolkit），依次点击确定。

● 安装pytorch 16:00

2. 安装pytorch

访问pytorch官网<https://pytorch.org/get-started/previous-versions/>

这个页面列出了GPU版本及对应的pytorch版本，并给出了对应的安装指令。打开命令后粘贴执行即可，安装时间会比较长。

例如：下图为适用于Linux、GPU12.6、GPU13.0、GPU13.2、CPU（对于没有GPU的情况）的安装指令。

```
LINUX AND WINDOWS
```

```
# ROCm 7.2 (Linux only)
pip install torch==2.12.1 torchvision==0.27.1 --index-url https://download.pytorch.org/whl/
# CUDA 12.6
pip install torch==2.12.1 torchvision==0.27.1 --index-url https://download.pytorch.org/whl/
# CUDA 13.0
pip install torch==2.12.1 torchvision==0.27.1 --index-url https://download.pytorch.org/whl/
# CUDA 13.2
pip install torch==2.12.1 torchvision==0.27.1 --index-url https://download.pytorch.org/whl/
# CPU only
pip install torch==2.12.1 torchvision==0.27.1 --index-url https://download.pytorch.org/whl/
```

安装完后输入python -c "import torch; print(torch.cuda.is_available()); print(torch.cuda.get_device_name(0))"，如果显示True并打印GPU信息，说明pytorch能正常工作。

3. 安装 Grounding DINO

在本地路径上新建一个文件夹，打开命令行并进入这个文件夹。

- 执行git clone <https://github.com/IDEA-Research/GroundingDINO> git 克隆仓库；
- 执行cd GroundingDINO 进入GroundingDINO文件夹；
- 执行pip install -e . 安装GroundingDINO，注意这一步可能会报错，多半是因为没有安装某个库导致，可以将错误信息发给AI智能体来解决问题。
- 执行mkdir weights 指令，新建weights文件夹；
- 从https://github.com/IDEA-Research/GroundingDINO/releases/download/v0.1.0-alpha/groundingdino_swint_ogc.pth 下载权重，放到weights文件夹。

4. 使用 Grounding DINO

○ 安装步骤:

- 访问 PyTorch官网。
- 根据之前确定的GPU版本，找到对应的安装指令并复制。
- 在命令行中粘贴执行指令，安装时间较长。

- 验证安装:

- 执行指令: `python -c "import torch; print(torch.cuda.is_available()); print(torch.cuda.get_device_name(0))"`。
- 若显示 True 并打印GPU信息, 说明PyTorch能正常工作。

- 安装GroundingDINO 16:42

3.安装 Grounding DINO

在任意路径上新建一个文件夹, 打开命令行并进入这个文件夹。

- 执行 `git clone https://github.com/IDEA-Research/GroundingDINO.git` 克隆仓库;
- 执行 `cd GroundingDINO` 进入GroundingDINO文件夹;
- 执行 `pip install -e .` 安装GroundingDINO, 注意这一步可能会报错, 多半是因为没有安装某个库导致, 可以将错误信息发给AI智能体来解决问题。
- 执行 `mkdir weights` 指令, 新建weights文件夹;
- 从https://github.com/IDEA-Research/GroundingDINO/releases/download/v0.1.0-alpha/groundingdino_swint_ogc.pth 下载权重, 放到weights文件夹。

4.使用 Grounding DINO

```
[1]: import os, sys
from pathlib import Path
import torch

GROUNDING_DINO_DIR = r"D:\DINO\GroundingDINO"
IMAGE_PATH = r"D:\DINO\GroundingDINO\input.jpg"
WEIGHTS_PATH = r"D:\DINO\GroundingDINO\weights\groundingdino_swint_ogc.pth"
CONFIG_PATH = rf"D:\DINO\GroundingDINO\config\groundingdino_swint_ogc.py"

sys.path.insert(0, GROUNDING_DINO_DIR)

[2]: from transformers.utils import is_offline_mode
from huggingface_hub import snapshot_download, model_info
import os

MODEL_NAME = "bert-base-uncased"

cache_dir = os.environ.get("HF_HOME", os.path.expanduser("~/.cache/huggingface"))
model_cache_path = Path(cache_dir) / "hub" / f"models--{MODEL_NAME.replace('/', '--')}"

if local_exists:
    os.environ["TRANSFORMERS_OFFLINE"] = "1"
    os.environ["HF_HUB_OFFLINE"] = "1"
else:
    os.environ["TRANSFORMERS_OFFLINE"] = "0"
    os.environ["HF_HUB_OFFLINE"] = "0"
```

可以在任意的路径上新建一个文件夹, 打开命令行工具并且进入。首先执行git

- 安装步骤:

- 在任意路径新建一个文件夹, 打开命令行并进入该文件夹。
- 执行 `git clone https://github.com/IDEA-Research/GroundingDINO.git` 克隆仓库。
- 执行 `cd GroundingDINO` 进入文件夹。
- 执行 `pip install -e .` 安装GroundingDINO (耗时较长)。
 - 易错点: 若报错, 多半是因为缺少某个库, 可将错误信息发给AI智能体分析解决。
- 执行 `mkdir weights` 新建权重文件夹。
- 从此链接 下载权重文件, 放入 weights 文件夹。

- 使用GroundingDINO 17:32

4.使用 Grounding DINO

```
[1]: import os, sys
from pathlib import Path
import torch

GROUNDING_DINO_DIR = r"D:\DINO\GroundingDINO"
IMAGE_PATH = r"D:\DINO\GroundingDINO\input.jpg"
WEIGHTS_PATH = r"D:\DINO\GroundingDINO\weights\groundingdino_swint_ogc.pth"
CONFIG_PATH = rf"D:\DINO\GroundingDINO\config\groundingdino_swint_ogc.py"

sys.path.insert(0, GROUNDING_DINO_DIR)

[2]: from transformers.utils import is_offline_mode
from huggingface_hub import snapshot_download, model_info
import os

MODEL_NAME = "bert-base-uncased"

cache_dir = os.environ.get("HF_HOME", os.path.expanduser("~/.cache/huggingface"))
model_cache_path = Path(cache_dir) / "hub" / f"models--{MODEL_NAME.replace('/', '--')}"

local_exists = model_cache_path.exists()
print(f"本地是否存在: {local_exists}")

if local_exists:
    os.environ["TRANSFORMERS_OFFLINE"] = "1"
    os.environ["HF_HUB_OFFLINE"] = "1"
else:
    os.environ["TRANSFORMERS_OFFLINE"] = "0"
    os.environ["HF_HUB_OFFLINE"] = "0"

本地是否存在: True

[3]: from groundingdino.util.inference import load_model
model = load_model(CONFIG_PATH, WEIGHTS_PATH)
```

FutureWarning: Importing from timm.models.layers is deprecated, please import via timm.layers
UserWarning: torch.meshgrid: in an upcoming release, it will be required to pass the indexing argument. (Triggered internally at C:\actions-runner\work\pytorch\pytorch\builder\windows\pytorch\aten\src\ATen\native\tensorshape.cpp:3596.)
final_text_encoder.type: bert-base-uncased
FutureWarning: You are using 'torch.load' with 'weights_only=False' (the current default value), which uses the default pickle module implicitly. It is possible to construct malicious pickle data which will execute arbitrary code during unpickling (See https://github.com/pytorch/pytorch/blob/main/SECURITY.md#untrusted-models for more details). In a future release, the default value for 'weights_only' will be flipped to 'True'. This limits the functions that can

- 导入并设置路径:

- 新建四个变量: 模型所在位置 (仓库路径)、输入图片路径、权重文件路径、配置文件路径。
- 使用 `sys.path.insert(0, GROUNDING_DINO_DIR)` 将仓库路径加入系统路径。

```

[1]: import os, sys
      from pathlib import Path
      import torch

      GROUNDING_DINO_DIR = "D:\DINO\groundingDINO"
      IMAGE_PATH = "D:\DINO\groundingDINO\input.jpg"
      WEIGHTS_PATH = "D:\DINO\groundingDINO\weights\groundingdino_swin_t_ogc.pth"
      CONFIG_PATH = "D:\DINO\groundingDINO\config\groundingdino_swin_t_ogc.py"

      sys.path.insert(0, GROUNDING_DINO_DIR)

[2]: from transformers.utils import is_offline_mode
      from huggingface_hub import snapshot_download, model_info
      import os

      MODEL_NAME = "bart-base-unccased"

      cache_dir = os.environ.get("HF_HOME", os.path.expanduser("~/cache/huggingface/"))
      cache_dir = Path(cache_dir) / "hub" / f"models--{MODEL_NAME.replace('/', '-')}"

      local_exists = model_cache_path.exists()
      if not local_exists:
          print(f"本地模型不存在, {local_exists}")

      is_offline_mode = os.environ["TRANSFORMERS_OFFLINE"] == "1"
      if not is_offline_mode:
          os.environ["HF_HUB_OFFLINE"] = "1"
      else:
          os.environ["TRANSFORMERS_OFFLINE"] = "0"
          os.environ["HF_HUB_OFFLINE"] = "0"

      本地是可用存在: True

```

sible to construct malicious pickle data which will execute arbitrary code during unpickling (See <https://github.com/pytorch/pytorch/blob/main/SECURITY.md#untrusted-models> for more details). In a future release, the default value for 'weights_only' will be flipped to 'True'. This limits the functions that can

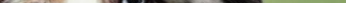
- ```
MODEL_NAME = "bert-base-uncased"

cache_dir = os.environ.get("HF_HOME", os.path.expanduser("~/.cache/huggingface"))
os.environ["HF_HOME"] = Path(cache_dir) / "hub" if "Models--" in MODEL_NAME.replace(".", "-") else ""

local_exists = model_cache_path.exists()
print(f"本地是否存在: {local_exists}")

if local_exists:
 os.environ["TRANSFORMERS_OFFLINE"] = "1"
 os.environ["HF_HUB_OFFLINE"] = "1"
else:
 os.environ["TRANSFORMERS_OFFLINE"] = "0"
 os.environ["HF_HUB_OFFLINE"] = "0"
```

```
[4]: from groundingdino.util.inference import load_image
 from PIL import Image
```



- final\_text\_encoder\_type: bert-base-uncased
- torchWarning: You are using 'torch.load' with 'weights\_only=False' (the current default value), which uses the deprecated pickle module implicitly. It is possible to construct malicious pickle data which will execute arbitrary code during unpickling (see <https://github.com/pytorch/pytorch/blob/main/SECURITY.md> for details). In a future release, the default value for 'weights\_only' will be flipped to 'True'. This limits the functions that can be executed during unpickling. Arbitrary objects will no longer be allowed to be loaded via this mode unless they are explicitly whitelisted by the user. It is recommended to manually set 'weights\_only=True' for any case where you do not have full control of the loaded file. Please open an issue on GitHub for any issues related to this experimental feature.

```
[4]: from groundingdino.util.inference import load_image
 from PIL import Image
```

```
image_source, image = load_image(IMAGE_PATH)
Image.open(IMAGE_PATH)
```



- 加载输入图片:

- 从 `groundingdino.util.inference` 导入 `load_image` 函数。
- 传入输入图片路径，加载图片并进行处理。



#### 核心推理:

- 设置文本提示词（如 "person . cat . dog"）。
  - **格式要求：** 类别之间必须用“空格.空格”分隔，格式错误会导致模型无法工作。
- 设置框置信度阈值（如0.35）和文本置信度阈值（如0.25）。
- 调用 `predict` 函数，传入模型、图像、提示词、框阈值和文本阈值。
- 返回三个结果：每个预测框的坐标、置信度、对应的文本。



#### 可视化结果:

- 使用 `annotate` 函数在图片上绘制检测框和标签。
- 使用 `matplotlib` 显示结果图片。
- 示例结果：检测到猫（置信度0.81）和狗（置信度0.62）。

#### 拓展作业:

- 思考如何利用GroundingDINO将图片中的狗替换成猫。
- **思路提示：** 先用GroundingDINO定位狗的位置并生成掩码，再用图像生成模型在掩码区域生成一只猫。

## 二、课程总结 21:45

### 1. 大模型与具身智能的核心区别



- 
- **大模型的核心能力:** 解决的是“看懂世界”的问题。
  - 侧重于对信息的理解、分析和认知。
- **具身智能的核心能力:** 解决的是“能动起来”的问题。
  - 侧重于在物理世界中的行动、交互和操作。
- **两者关系:** 大模型提供“大脑”（认知），具身智能提供“身体”（行动），二者结合才能实现完整的智能体。

### 三、知识小结

| 知识点               | 核心内容                                                                                     | 考试重点/易混淆点                                          | 难度系数  |
|-------------------|------------------------------------------------------------------------------------------|----------------------------------------------------|-------|
| <b>多模态模型定义</b>    | 让同一模型将文本、图像、语音、视频、传感器等信息从语义层面进行对齐，融合成判断                                                  | 核心是 <b>对齐和推理</b> ，而非多输入；单模态是专项工具，多模态是综合助手          | ★★★★☆ |
| <b>多模态模型代表</b>    | GPT-4o（原生全模态，端到端融合）、DeepSeek-VLM（开源，细粒度理解，中文优化）、千问3.5O-mini（混合注意力MoE架构，音视频理解+Web coding） | GPT-4o是 <b>拼接式</b> vs 原生融合；千问3.5O-mini抛弃外挂编码器      | ★★★★☆ |
| <b>聚生智能定义</b>     | 有物理或虚拟身体的AI形态（机器人、自动驾驶、无人机、游戏NPC），完成一整套动作而非仅回答问题                                         | 普通AI回答问题，聚生智能 <b>完成动作</b> （感知→理解→规划→行动→反馈）         | ★★★★☆ |
| <b>聚生智能核心技术</b>   | VLA（Vision-Language-Action）模型：连接感知、语言、动作；世界模型：在数字空间预演动作后果                                | VLA是 <b>身体</b> ，世界模型是 <b>灵魂</b> ；反应式 vs <b>预判式</b> | ★★★★★ |
| <b>多模态与聚生智能融合</b> | 多模态大模型提供超级感官（看懂世界），聚生智能提供物理化身（能动起来），互动中反馈反哺模型                                            | 光有多模态是 <b>纸上谈兵</b> ，光有聚生智能是 <b>四肢发达头脑简单</b>        | ★★★★☆ |



|                  |                                             |                                      |        |
|------------------|---------------------------------------------|--------------------------------------|--------|
| Grounding DINO工具 | 文本驱动零样本目标检测，输入文本提示词（如“cat, dog”）即可检测并标注     | 输入格式严格：类别后跟空格点空格；需配置GPU、PyTorch、权重文件 | ★★★★☆☆ |
| 拓展作业             | 利用Grounding DINO定位狗的位置生成掩码，再用图像生成模型在掩码区域生成猫 | 结合检测+生成两阶段实现图像编辑                     | ★★★★☆☆ |

|      |                                 |                  |                        |
|------|---------------------------------|------------------|------------------------|
| 对比维度 | 多模态模型                           | 聚生智能             | 融合价值                   |
| 核心能力 | 看懂世界（感知+理解）                     | 能动起来（规划+执行）      | 从数字到物理的闭环              |
| 技术瓶颈 | 语义对齐与推理                         | 物理世界规律建模         | 反馈反哺提升真实世界理解           |
| 代表应用 | GPT-4o、DeepSeek-VLM、千问3.5O-mini | 人形机器人、自动驾驶、游戏NPC | 北京人形机器人半马（2025→2026进化） |
| 发展阶段 | 从拼接式到原生融合                       | 从反应式到预判式         | 一年内从“学跑步”到“会跑步”        |